

Security Testing CTF Lab Report

10-Level Penetration Testing Capture-The-Flag Challenge

Aditi Vaidyanathan

March 2025

Level 1:

I began by inspecting the login page using the Browser Developer Tools and noticed that the login button was disabled by default. To enable it, I executed the following command in the console:

```
document.getElementById("myLogin").disabled = false;
```

While reviewing the page's source code, I found a comment on line 15 that revealed test credentials—specifically, the username `test-user` and password `test`. Logging in with these credentials worked, but it became clear this was only a placeholder account, as no further progress could be made.

Upon digging deeper, I suspected that a different user might be required. I used the following JavaScript command to change the value of the username field:

```
document.querySelector('input[name="username"]').value = "admin";
```

After entering the password `warmingup`, I was successfully logged in, confirming that `admin` was the intended account to complete this level of the challenge.

Level 2:

I started off the second challenge by inspecting the associated JavaScript files and quickly noticed that the login script didn't actually validate the username or password. This meant that any set of fake credentials could be used to bypass the login step.

After logging in with dummy credentials, I was presented with a message stating that requests were only accepted from an Italian server. Upon examining the HTML and JavaScript more closely, I found that the script was checking whether the incoming request originated from an Italian domain. Realizing this, I decided to spoof the `Referer` header to trick the server into thinking my request came from an Italian domain. I began researching how to implement this approach and learned how to craft a custom `POST` request using `curl` in the terminal. I constructed a request using made-up credentials and included a `Referer` header set to `http://example.it`, a generic Italian domain.

Despite several attempts, I wasn't successful at first. Eventually, I realized the issue was that I was targeting the wrong endpoint. The correct URL for the request was `/cgi-bin/verify.js`. Once I adjusted my request accordingly, the final payload worked, and I successfully received the link to the next challenge. The final payload that I used to complete this challenge is as follows:

```
curl -X POST -e "http://example.it" --data "username=4lph4&password=heyyy"  
"192.168.52.97:62935/jaejoy8Keeg7iNag/cgi-bin/verify.js"
```

Level 3:

Upon viewing the hint, which indicated that the challenge may be solved via Reflected XSS, I then used an XSS test in the "Enter your name" field by inputting this simple XSS payload:

```
<script>alert("XSS Test")</script>
```

An alert box appeared stating “XSS Test,” thereby indicating that the input is vulnerable to XSS. To perform a Reflected Cross-Site Scripting (XSS) attack, I placed the following script inside the "Enter your name" field and clicked Submit: `<script>alert(document.body.innerHTML)</script>`

As mentioned in the hint, the website is vulnerable to Reflected XSS, and an alert box popped up containing the entire HTML content of the page. From the extracted script within the alert box pop-up, I noticed the following line: `const obf_flag = "aHR0cDovLzE5Mi4xNjguNTIuOTc6NjMwMzQvYWZhNWVIR2hvaGcyZXRIQ=="`

I also realised that the `atob(obf_flag)` function was being used, which means the flag is encoded in Base64. On the browser’s console tab, I placed the following: `console.log(atob("aHR0cDovLzE5Mi4xNjguNTIuOTc6NjMwMzQvYWZhNWVIR2hvaGcyZXRIQ==" ));`

Upon pressing Enter, it revealed the flag URL. However, I realised I was overcomplicating the challenge and went back to the very beginning to properly understand the objective. The challenge required executing JavaScript via Reflected XSS to reveal the hidden flag. The flag is hidden inside an obfuscated JavaScript function (`printFlag()`), which can be executed. Since the flag is hidden in `printFlag()`, the goal is to execute that function using an XSS payload. Finally, I used the following payload: `<script>printFlag();</script>` to obtain the personalised URL to Level 4.

Level 4:

Before starting the challenge, I reviewed the hint provided, which suggested that this was a Command Injection attack. To begin, I started by inspecting the HTML page and quickly noticed that the JavaScript was taking user input directly from the `filter` parameter and appending it to a request sent to a PHP script on the server.

Further inspection revealed that there was no input validation on the `filter` parameter, meaning user input could potentially be passed directly into a shell command.

I decided to use `curl` to craft a payload that would execute the `ls` command, which lists files in a directory. However, when I ran the request, the terminal returned no user or visible output, indicating the payload wasn’t working as intended. I revisited the hint and noticed a reference to the `Referer` approach. To understand this better, I re-inspected the HTML and discovered that server-side scripting was checking the `Referer` field of the HTTP request as a validation step. With this in mind, I created a new `curl` payload and added a custom `Referer` header in the request. Unfortunately, I still received the same empty response.

I turned to the lecture notes for more insight and realized the issue was with how I was separating commands. I had used a semicolon (;) directly, which the system likely detected and blocked. To bypass this, I encoded the semicolon as `%3B`, which is the URL-encoded version of ;. This was a crucial step, as it allowed me to terminate the existing command and append my own (`ls`). The vulnerability was confirmed when the directory listing was successfully returned.

To test and confirm the command injection, I used the following final `curl` command to list the current directory contents:

```
curl -H "Referer: "http://192.168.52.97:63034/ahs5eeGhohg2etee/"  
"http://192.168.52.97:63034/ahs5eeGhohg2etee/cgi-bin/users.php?filter=;ls"
```

After the command executed successfully, I discovered a file named `secretuser.txt`, which I believed might contain the flag. To confirm this, I updated my command to read the file's contents using:

```
curl -H "Referer: "http://192.168.52.97:63034/ahs5eeGhohg2etee/"  
"http://192.168.52.97:63034/ahs5eeGhohg2etee/cgi-bin/users.php?filter=%3Bcat%20secretuser.txt"
```

The contents of this `secretuser.txt` file revealed the link to Level 5 of the challenge.

Level 5:

At first glance, I assumed the vulnerability was related to SQL injection, so I tested the classic payload `"or 1=1"`. I initially tried it in the username field, but it didn't work. I received the same result when testing the password field, which confirmed that the application was not vulnerable to SQL injection. I then visited the hint URL to gather more information that could guide me forward. After reviewing the challenge's hints, I returned to the login page and began testing common default credentials. Eventually, the combination `admin:admin` worked, successfully logging me into the next page. This suggested that the application was vulnerable due to weak authentication mechanisms.

Once I gained access, I used the browser's inspect tool to search for any clues that could help me progress, but after thoroughly examining the page, I couldn't find anything that led to the next challenge. However, I noticed that the URL included a `uid` parameter, which appeared to be linked to user accounts. I suspected an Insecure Direct Object Reference (IDOR) vulnerability—meaning that modifying the `uid` value might reveal information from other users' accounts. I manually altered the `uid` parameter in the URL to different values between 0 to 5, and each time, it granted me access to another user's page. Finally, when I changed `uid=6`, I found a hidden entry that contained a URL to level six of the challenge. This ultimately confirmed that the application lacked proper access controls, allowing unauthorized access to other users' data.

Level 6:

I began this level by checking the hints for the challenge, as I was not able to understand what the objective was. Then, I tested various username and password combinations to observe how the system would respond. When I entered an incorrect username, it returned "wrong username," but when I tried the username **Luca**, it responded with "wrong password." This indicated that **Luca** was a valid user, which suggested I might be able to impersonate him.

I then logged in as a guest user and used **Burp Suite** to inspect the cookies. I quickly understood that this is a cookie-based authentication manipulation challenge. I noticed a cookie named **u**, which appeared to store the currently logged-in user's name. Initially, I tried manually changing the cookie value to **Luca**, but that didn't work. Recalling the hint from Prof. Pierazzi, who mentioned **ROT13 encoding**, I decided to try that approach. I encoded **Luca** using ROT13, which resulted in **yhpn**, and updated the cookie value in Burp Suite accordingly. Upon refreshing the page, I successfully found the link and gained access to Level 7 of the coursework.

Level 7:

At the beginning of this level, I referred to the hint and quickly understood that it would be easier if I used the SQLMap tool, which is useful for automating SQL injection attacks. I began by experimenting with various fields in the New User registration form to identify any potential vulnerabilities.

Through testing, I discovered that the username field was susceptible to SQL injection payloads. To verify this, I manually entered a simple payload:

```
sqlmap -u  
"http://192.168.52.97:62809/ahh1eeWiemohbee7/cgi-bin/process_createuser.php"--data="firstname=test  
&lastname=test&email=test%40test&username=test&password=test" -p username --method POST  
--dump --dbs
```

I wrote this SQLMap payload to attempt an SQL injection specifically on the **username** field of the user registration form. The **-p username** option directs SQLMap to focus the injection only on that parameter, ignoring the rest of the form fields.

The command sends a POST request to the target URL using the data typically submitted during user registration—**firstname**, **lastname**, **email**, **username**, and **password**. These fields are filled with test values and passed using the **--data** flag, while **--method POST** ensures the correct request type is used.

My goal with this payload was 3 fold:

- **Perform an SQL injection on the **username** field** to test for vulnerabilities.
- **Dump the output** using the **--dump** option, in case the injection was successful.
- **List all the databases found on the server** via the **--dbs** flag, allowing me to explore the server's database structure further.

As the hint suggested, I discovered user credentials for both Kevin and Tsutomu. I then logged in using Tsutomu's account and began exploring his blog. In his "My First Blog" post, I came across the hidden link that led me to the next challenge.

Level 8:

I started off this level by checking the hint page, which indicated that I needed to impersonate Tsutomu again, similar to what was required in the previous challenge. Initially, I filled out the form with the three required values and received a message stating that the password had been saved. To better understand what was happening behind the scenes, I opened Developer Tools for further inspection.

While inspecting the page, I discovered a hidden input field named `admin`, which was set to `0`. Using Burp Suite, I intercepted the request and modified it by adding `admin=1` before forwarding it to the server. After submitting the altered form, I was redirected to a blank page with no cookies, which suggested that I also needed to modify the cookies to proceed.

I switched to Burp Suite's Repeater feature to test various cookie configurations. My first attempt involved encoding Tsutomu's name in ROT13, as I had done in the previous challenge, but this didn't produce the desired result. Subsequently, I tried using Tsutomu's full name in plain text and updated the cookie as follows:

Cookie: `user=Tsutomu Shimomura; admin=1` ; Once I sent the request with this updated cookie and refreshed the page, the link to the next challenge was successfully revealed.

Level 9:

I began level 9 by reviewing the hint page, which referenced environment variables and recommended using Burp Suite to bypass JavaScript-based input filtering. While examining the page with Developer Tools, I identified a sanitization script that blocked specific characters using the pattern `/[:{};]/g`, making it impossible to inject commands directly. I noticed that the `filter` parameter in the request to `GET /oiciCieH7to0eigo/cgi-bin/env.php?filter=` was modifiable, so I experimented with various inputs to observe how the system would respond.

Initially, I tried multiple terminal commands in hopes of triggering a response or revealing any files, but none of them worked. After several unsuccessful attempts, I revisited the LGT3 lecture slides from my coursework and realized that the challenge was actually based on a Shellshock vulnerability. Since the direct input was being filtered, I had to craft a URL-encoded version of the payload to bypass the restrictions. I constructed the following encoded Shellshock payload:

```
%28%29%20%7b%20%3a%3b%7d%3b%20ls
```

In a nutshell, this challenge turned out to be a disguised Shellshock attack, which allowed me to execute the `ls` command and list the files on the server. The output revealed a hidden file named `secret4uouO.txt`. Eventually, I modified my payload to read the contents of that file using:

```
cat:%28%29%20%7b%20%3a%3b%7d%3b%20cat%20secret4uouO.txt
```

This command successfully returned a hidden link leading to the next challenge. Overall, through this challenge, I learned how Shellshock can be exploited to execute arbitrary commands on a vulnerable server, and how insufficient input validation can expose a system to serious risks.

Level 10:

I began the final level by inspecting the page and noticed a file named `PacMan.class`. Unfortunately, since modern browsers no longer support Java files, I had to download the file manually in order to analyze it. To do this, I used the following command to retrieve the hidden `PacMan.class` file from the server: `curl http://192.168.52.97:63227/xa7Aiceigiom8pee/PacMan.class --output PacMan.class`

Since the Java `.class` files were compiled, I needed to decompile the file to examine the source code. I initially attempted to use the Procyon decompiler but encountered issues with downloading it. As a result, I switched to a different Java decompiler by running the following terminal commands: `sudo apt update`, `sudo apt install jd-gui`

Eventually, I used JD-GUI to decompile and inspect the `.class` file. While reviewing the code, I came across a string variable named `msg1` on line 91, which matched the level's hint. This led me to the next step, which was to encode the message I had found. Upon analysing the `.class` file further, I found a function call on line 668, which led me to `decrString()` function. This function contained a java decryption function. I rewrote the script in Python for ease in understanding and execution.

The re-written python script is as follows:

```
def decr_string(param_string):
    array_of_char = [""] * len(param_string)

    for b in range(len(param_string)):
        array_of_char[b] = chr(((ord(param_string[b]) - 32 + b * 42) % 95) + 32)

    return "".join(array_of_char)

encrypted_string = "hJ QPzP(e4e>xP{X+\\=pIzL$U/_p={LO[+bi#Sf! Y5"
decrypted_string = decr_string(encrypted_string)
print(decrypted_string)
```

Upon executing the Python code, the output revealed the link to the Victory page of the CTF challenge. The victory link that I obtained was: <http://192.168.52.97:62404/jaje3ice7zuS8aek/>

Overall Considerations:

Throughout this coursework, I believe I have gained valuable hands-on experience in identifying and exploiting a wide range of web application vulnerabilities. Personally, having some prior knowledge in this subject matter definitely helped, but I still learned a lot by working through each challenge. These challenges covered a broad set of topics, including credential manipulation, header spoofing, cross-site scripting (XSS), command injection, insecure direct object references (IDOR), cookie-based authentication flaws, SQL injection, Shellshock exploitation, and even decompiling Java applets. I felt

each level was unique in its own way, with some tasks pushing me to think outside the box and get a bit creative. This made the coursework quite rewarding!

Many challenges required me to dig deeper into the topic or research alternative solutions based on the hints provided. This not only helped me improve technically but also motivated me to stay on top of the weekly readings, revisit course content, and better prepare for the upcoming term-end exams. Whenever the levels felt a little too challenging and I got stuck, I often found parts of the solution within the course materials, which kept me motivated to push through and complete the coursework efficiently.

In my experience, the most difficult challenges I encountered were Challenges 3 and 9. These took me the longest to solve. In Challenge 3, I initially misinterpreted the hint and didn't follow through correctly until much later, when I realised I was overcomplicating the process. This taught me an important lesson: think simply—out-of-the-box questions don't always require complex solutions.

Challenge 9, on the other hand, was tough because the hints were minimal, and I wasn't entirely sure how to approach it. I spent a lot of time experimenting with different terminal commands, hoping one of them would eventually reveal the system files I needed to proceed. Despite several attempts, I made little progress. Eventually, I turned to the discussion threads on the forum and was encouraged to revisit the course materials. That's when I found the LGT3 slides particularly helpful—they gave me a clearer understanding of how to structure my payload. The system had input sanitisation in place, blocking characters like colons and semicolons, so I had to get creative by using URL encoding to bypass the restrictions. After carefully crafting a new payload, I was finally able to execute the command and access the secret file.

Overall, these challenges significantly strengthened my critical thinking and problem-solving abilities in the cybersecurity domain. They taught me the importance of careful inspection, proper validation, and rigorous security testing—skills that are essential for securing modern web applications.